

Complete Guide: Binary Tree Implementation Strategies in Python

When implementing Binary Trees in Python, developers must choose strategies based on memory constraints, traversal requirements, and structural rules. This guide outlines the two primary representation strategies—**Linked Structures** and **Array-Based Representations**—and details how to navigate them.

1. The Linked Structure Strategy

The Linked Structure is the most common way to build dynamic trees. Each node is an independent object containing the stored data (element) and pointers to other nodes.

A. Simple vs. Bidirectional Pointers

In a basic setup, nodes only point downwards to their children. To enable upstream navigation, we introduce a **parent pointer**. This is crucial for advanced algorithms like finding an in-order successor or climbing back up the tree during self-balancing rotations.

Standard Dynamic Node with Parent Reference:

```
class Node:
    """Lightweight, non-public class for storing a node in a binary tree."""
    def __init__(self, element, parent=None, left=None, right=None):
        self.element = element
        self.parent = parent # Link to the parent node
        self.left = left     # Link to the left child
        self.right = right   # Link to the right child
```

B. Enterprise Object-Oriented Architecture

In professional codebases, trees are constructed using inheritance. We define an **Abstract Base Class (ABC)** to establish a unified API, and then inherit from it to build concrete structures like `LinkBinaryTree` or `AVLTree`.

```
from abc import ABC, abstractmethod

class BinaryTree(ABC):
    """Abstract base class representing a binary tree structure."""

    @abstractmethod
    def root(self):
        """Return the root position of the tree (or None if empty)."""
        pass

    @abstractmethod
    def left(self, p):
        """Return the position of p's left child (or None if empty)."""
        pass

    @abstractmethod
    def right(self, p):
```

```

    """Return the position of p's right child (or None if empty)."""
    pass

def sibling(self, p):
    """Return the position of p's sibling (or None if no sibling)."""
    parent = p.parent
    if parent is None:
        return None # p is the root
    if p is parent.left:
        return parent.right
    else:
        return parent.left

class LinkedBinaryTree(BinaryTree):
    """Concrete implementation of a binary tree using a linked structure."""

    def __init__(self):
        self._root = None
        self._size = 0

    def root(self):
        return self._root

    def left(self, node):
        return node.left

    def right(self, node):
        return node.right

    def add_root(self, element):
        """Insert a root node into an empty tree."""
        if self._root is not None:
            raise ValueError("Root already exists")
        self._size = 1
        self._root = Node(element)
        return self._root

    def insert_left(self, node, element):
        """Create a new left child for the given node."""
        if node.left is not None:
            raise ValueError("Left child already exists")
        self._size += 1
        node.left = Node(element, parent=node)
        return node.left

    def insert_right(self, node, element):
        """Create a new right child for the given node."""
        if node.right is not None:
            raise ValueError("Right child already exists")
        self._size += 1
        node.right = Node(element, parent=node)
        return node.right

```

2. The Array-Based Representation Strategy

For **complete binary trees** (where every level except possibly the last is completely filled, and all nodes are as far left as possible), pointer-based trees introduce unnecessary memory overhead.



Instead, we can flatten the tree into a standard Python list.

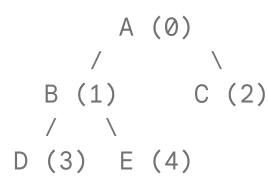
A. The Core Mathematics

If a node's index in the list is represented by i , we can navigate the tree entirely using fast arithmetic instead of memory references:

- **Left Child Index:** $2i + 1$
- **Right Child Index:** $2i + 2$
- **Parent Index:** $(i - 1) // 2$ (using Python's integer/floored division)

Computational Example:

Suppose we have the list representation: ["A", "B", "C", "D", "E"]



To find the parent of E (index 4):

$$\text{Parent Index} = (4 - 1) // 2 = 3 // 2 = 1$$

Index 1 corresponds to B, which is correct.

B. Real-World Use Case: Ahnentafel (Genealogical Tables)

This sequential array layout is historically identical to **Ahnentafel** systems used in genealogy. Since every individual has exactly two biological parents, your ancestors form a perfect, gapless binary tree. You can store generations of family history in a flat list without pointers, calculating relationships dynamically using the array index formulas.

3. Tree Navigation & Traversal Strategies

Because trees are non-linear, we must use specific algorithms to visit nodes systematically.

A. Depth-First Search (DFS)

DFS dives deep down a branch before backtracking. The output order depends on when the parent node is evaluated relative to its children:

1. **Pre-order** ($Root \rightarrow Left \rightarrow Right$): Used for copying/cloning trees.
2. **In-order** ($Left \rightarrow Root \rightarrow Right$): Displays elements of a Binary Search Tree (BST) in sorted order.
3. **Post-order** ($Left \rightarrow Right \rightarrow Root$): Perfect for bottom-up calculations (e.g., measuring total directory size or deleting a tree safely).

```
def traverse_in_order(node):
    """In-order traversal: Left, Root, Right"""
```

```
if node is not None:
    traverse_in_order(node.left)
    print(node.element, end=" ")
    traverse_in_order(node.right)
```

B. Breadth-First Search (BFS / Level-order)

BFS visits nodes level-by-level, from top to bottom and left to right. Because recursion uses a stack structure, BFS must be implemented **iteratively** using a **Queue** (`collections.deque` in Python) to preserve FIFO (First-In, First-Out) ordering.

```
from collections import deque

def traverse_level_order(root_node):
    """BFS traversal: Level by level, Left to Right"""
    if root_node is None:
        return

    queue = deque([root_node])
    while queue:
        current = queue.popleft()
        print(current.element, end=" ")

        if current.left:
            queue.append(current.left)
        if current.right:
            queue.append(current.right)
```

4. Comparison Matrix

Strategy	Memory Overhead	Insertion Speed	Best For
Linked Structure	High (storing 3 pointers per node)	$O(1)$ (just updating references)	Unbalanced, sparse, or highly dynamic trees
Array Representation	Low (no pointer objects stored)	$O(1)$ amortized (appending to list)	Complete, dense trees (e.g., Binary Heaps, Priority Queues)